LAPORAN

SOFTWARE CONSTRUCTION

Implementasi Service Pangkalan pada Sistem PORTOM (Peta Pangkalan Transportasi Umum)

Oleh:

Rahmadita (232410103037)

PRODI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS JEMBER

2026

DAFTAR ISI

DAFTAR ISI 2
1. Deskripsi layanan 3
2. Endpoint API (CRUD) 4
2.1. GET - Ambil Data 4
2.2. POST - Tambah Data 5
2.3. PUT - Update Data 6
2.4. DELETE - Hapus Data 8
3. Kesimpulan 9
4. Link Github 10

1. Deskripsi layanan

Service Pangkalan adalah bagian sistem dari PORTOM (Peta Pangkalan Transportasi Umum) yang tujuannya untuk mengelola data lokasi pangkalan transportasi umum seperti angkot, becak, dan bajaj.

Layanan Portom dibuat menggunakan Node.js dengan framework Express dan Sequelize ORM untuk menyambungkan aplikasi ke database MySQL.

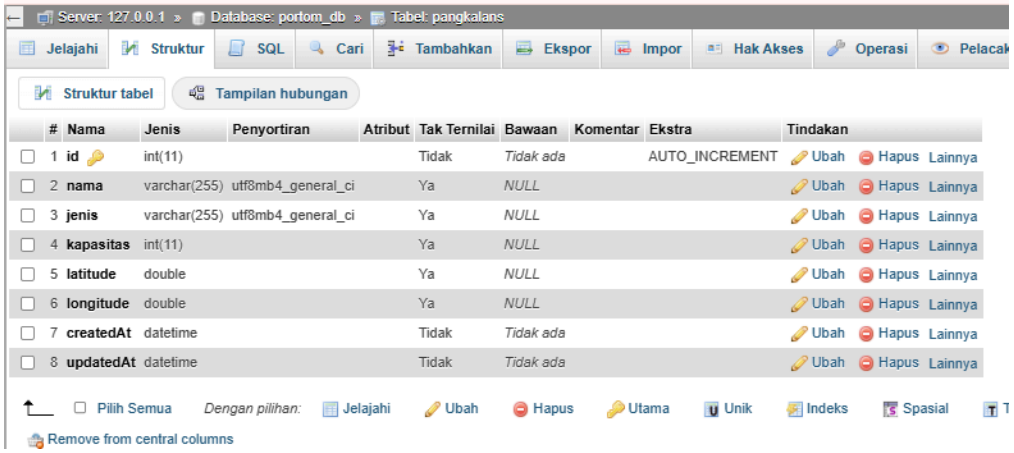
Fungsi layanan ini untuk melakukan operasi CRUD (Create, Read, Update, Delete) terhadap data pangkalan.

a. Teknologi yang Digunakan

1.) Node.js
2.) Express.js
3.) Sequelize ORM
4.) MySql (phpMyAdmin)
5.) Postman

b. Struktur data

Tabel yang digunakan adalah tabel Pangkalan dengan atribut seperti dibawah ini:



1.) id merupakan primary key dengan tipe integer akan diisi secara otomatis sebagai pengenal unik setiap data pangkalan.
2.) nama merupakan nama pangkalan dengan tipe varchar(255), untuk menyimpan nama lokasi pangkalan transportasi umum.
3.) jenis merupakan jenis kendaraan dengan tipe varchar(255), untuk menunjukkan kategori transportasi yang beroperasi di pangkalan tersebut, seperti angkot, becak, dan juga bajaj.

4.) kapasitas merupakan jumlah kapasitas kendaraan dengan tipe integer, untuk menggambarkan seberapa banyak kendaraan yang dapat ditampung di pangkalan.

5.) latitude merupakan koordinat lintang dengan tipe double, digunakan untuk menentukan posisi geografis pangkalan pada peta.

6.) longitude merupakan koordinat bujur dengan tipe double, digunakan bersama latitude untuk menentukan lokasi pangkalan secara akurat.

7.) createAt merupakan waktu pembuatan data bertipe datetime, diisi otomatis oleh Sequelize saat data pertama kali ditambahkan.

8.) updateAt merupakan waktu pembaruan data bertipe datetime, diperbarui otomatis oleh Sequelize setiap kali data diubah.

2. Endpoint API (CRUD)

2.1. GET - Ambil Data

a. Kode Implementasi

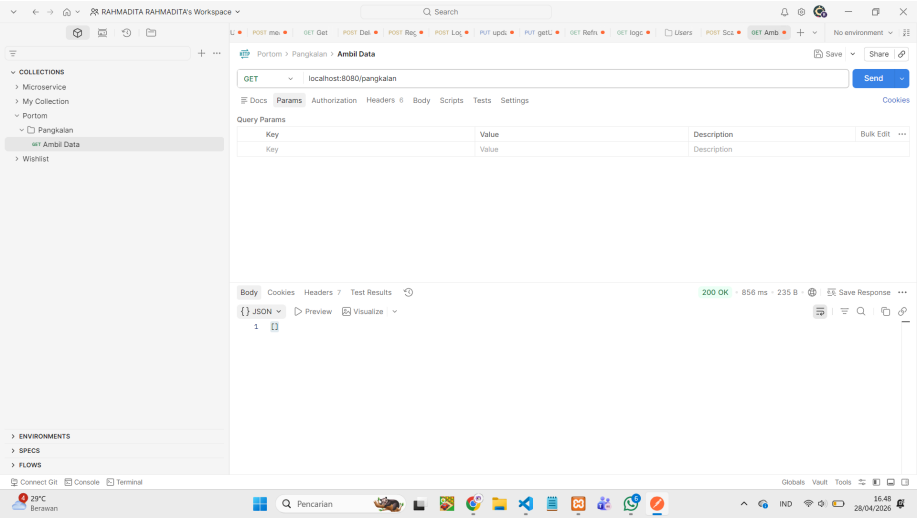
```
router.get('/', async (req, res) => {
  try {
    const data = await Pangkalan.findAll();
    res.json(data);
  } catch (error) {
    res.status(500).json({
      message: 'Terjadi kesalahan',
      error
    });
  }
});
```

b. Endpoint:

```
GET /pangkalan
```

Digunakan untuk mengambil seluruh data pangkalan dari database.

c. Hasil Pengujian (Postman):



Berhasil menampilkan data pangkalan dalam bentuk array JSON.

2.2. POST - Tambah Data

a. Kode Implementasi

```
router.post('/', async (req, res) => {
    try {
        const { nama, jenis, kapasitas, latitude, longitude
        } = req.body;

        const newData = await Pangkalan.create({
            nama,
            jenis,
            kapasitas,
            latitude,
            longitude
        });
        res.status(201).json(newData);
    } catch (error){
        res.status(500).json({
            message: 'Gagal menambahkan data',
        });
    }
});
```

b. Endpoint:

```
POST /pangkalan
```

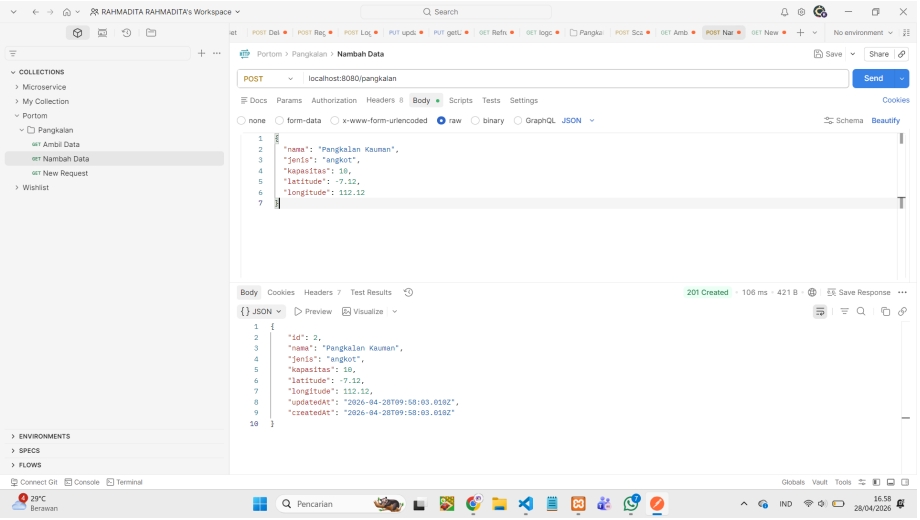
c. Body (JSON):

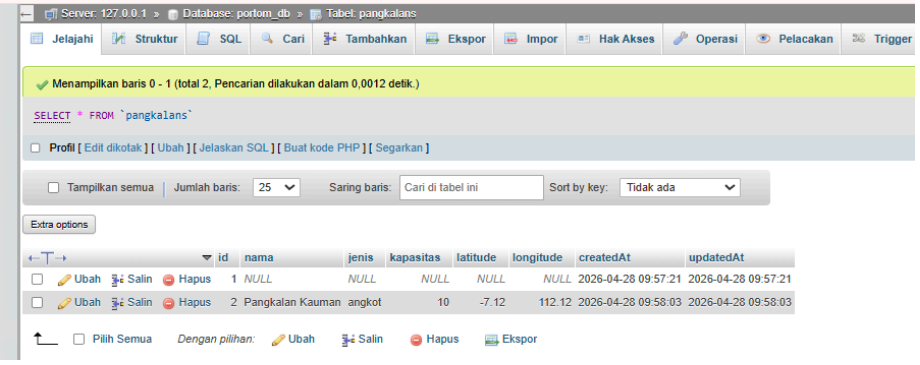
```
{
  "nama": "Pangkalan Kauman",
  "jenis": "angkot",
```

```
  "kapasitas": 10,
  "latitude": -7.12,
  "longitude": 112.12
}
```

Digunakan untuk menambah data pangkalan baru.

d. Hasil Pengujian (Postman):





Berhasil menambahkan data baru ke dalam database.

2.3. PUT - Update Data

a. Kode Implementasi

```
router.put ('/:id', async (req, res) => {
    try {
        const{id} = req.params;
        const{ nama, jenis, kapasitas, latitude, longitude
        } = req.body;

        const data  = await Pangkalan.findByPk(id);

        if(!data) {
            return res.status(404).json({
```

```
                message: 'Data tidak ditemukan'
            });
        }
        await data.update({ nama, jenis, kapasitas, latitude,
longitude
        });
        res.json(data);
    } catch (error) {
        res.status(500).json({
            message: 'Gagal update data',
            error
        });
    }
});
```

b. Endpoint:

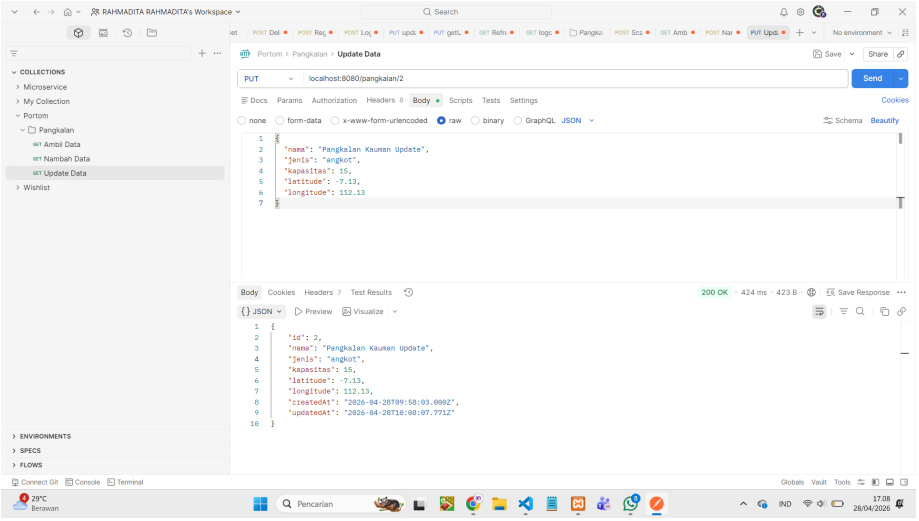
```
PUT  /pangkalan/:id
```

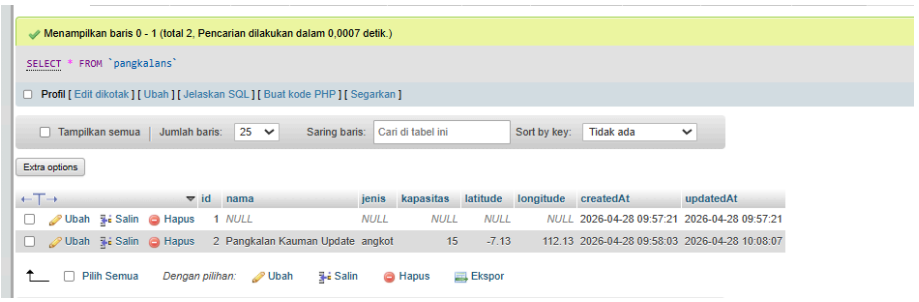
c. Body (JSON):

```
{
  "nama": "Pangkalan Kauman Update",
  "jenis": "angkot",
  "kapasitas": 15,
  "latitude": -7.13,
  "longitude": 112.13
}
```

Digunakan untuk memperbarui data pangkalan berdasarkan ID.

d. Hasil Pengujian (Postman)





Berhasil memperbarui data pangkalan yang sudah ada.

2.4. DELETE - Hapus Data

a. Kode Implementasi

```
router.delete('/:id', async (req, res) => {
    try {
        const{id} = req.params;

        const data = await Pangkalan.findByPk(id);

        if(!data) {
            return res.status(404).json({
                message: 'Data tidak ditemukan'
            });
        }

        await data.destroy();

        res.json ({
            message: 'Data berhasil dihapus'
        });
    } catch (error) {
        res.status(500).json({
            message: 'Gagal menghapus data',
```

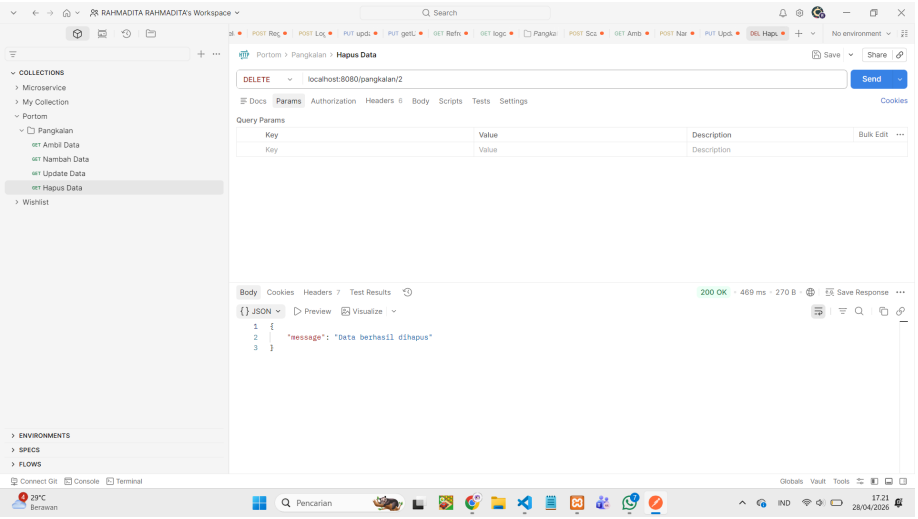
```
            error
        });
    }
});
```

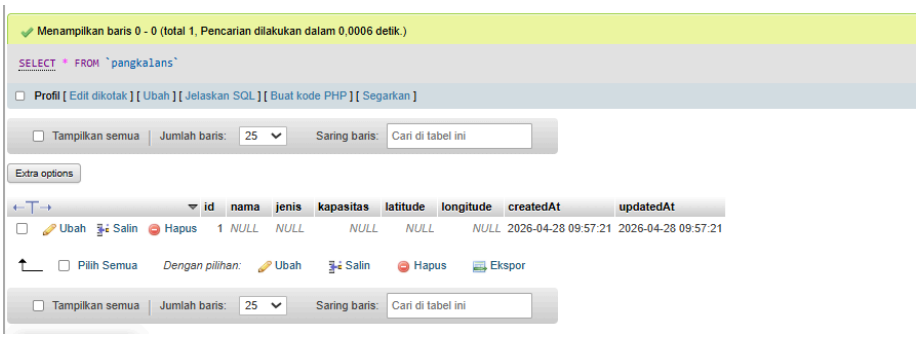
b. Endpoint:

```
DELETE /pangkalan/:id
```

Digunakan untuk menghapus data pangkalan berdasarkan ID.

c. Hasil Pengujian (Postman):





Berhasil menghapus data dari database.

3. Kesimpulan

Service Pangkalan berhasil dibangun menggunakan Express dan Sequelize dengan implementasi operasi CRUD yang lengkap.

Layanan ini digunakan untuk mempermudah masyarakat dalam mengetahui lokasi pangkalan transportasi umum serta mendukung pengurangan kemacetan dan polusi.

4. Link Github

232410103037_Rahmadita_Portom